

2025_CyKor_Week1

1. Push & Pop

```
void push(char* info, int value)
{
    SP++;
    strcpy(stack_info[SP], info);
    call_stack[SP] = value;
}

int num_pop;
void pop()
{
    SP--;
}

// ex)
// func2(11, 13);

// func2의 스택 프레임 제거 (함수 에필로그 + pop)
// epillog();
// for (num_pop = 0; num_pop < 4; num_pop++){pop();}
```

먼저 Push 와 Pop을 구현하였다. Stack은 원래 쌓일 수록 SP의 값이 줄어들지만 이번 과제에서는 쌓일 수록 SP 값이 커지기 때문에 실제 주소값의 변화와 반대로 될 것이다. Push 에는 call_stack에 들어갈 value 값과 info에 들어갈 문자열을 입력받아 어셈블리어인 `sub esp, 4 , mov [esp], val` 를 그대로 시행하였다.

Pop에서는 실제로 `pop (reg)` 로 그 자리에 있던 값을 복사하는 행동을 하지만 이 과제에서는 pop 기능을 사용한다기보다는 스택을 정리하는 프로로그 과정에서 pop을 사용하기에 값을 따로 복사해놓지는 않고 SP를 줄였다. caller에서 스택 프레임을 제거하기 때문에 num_pop 변수와 for 문으로 스택을 정리하는 것을 나타내었다.

2. Epilog & Prolog

```

void epililog(){
    SP = FP;
    FP = call_stack[SP];
}

void func2(int arg1, int arg2)
{
    int var_2 = 200;

    // func2의 스택 프레임 형성 (함수 프로로그 + push)
    push("func2 SFP", FP);
    FP = SP;
    push("var_2", var_2);
    .....
}

```

Epilog는 `mov esp, ebp(fp)` , `pop ebp` 이지만 이것은 `ret` 전에 함수를 정리하기 때문에 가능하지만 과제에서는 caller에서 함수를 정리하기 때문에 일단 이전 함수의 `sp`와 `fp`를 회복하는 기능으로 만들어 놓았다.

Prolog는 callee에서 하고 있는데 SFP의 info를 입력하는 과정에서 함수 이름이 들어가 따로 함수를 만들지는 않고 함수 내부에 코드로 작성하였다.

3. Function Calling

```

push("arg3", 3);
push("arg2", 2);
push("arg1", 1);
push("Return Address", -1);
func1(1, 2, 3);

```

함수를 부를 때에는 caller에서 인자를 오른쪽부터 push하며 마지막에 return 주소를 push한다. 이후 함수 안으로 들어가 프로로그가 일어나며 `ret` 위에 `sfp`가 덮어씌여지게 될 것이다.

4. 추가 구현

추가 구현을 하기 위해 스택 주소와 실제 코드의 주소를 넣으려했으나

```

pwndbg> disass func3
Dump of assembler code for function func3:
0x00001552 <+0>:    push    ebp
0x00001553 <+1>:    mov     ebp,esp
0x00001555 <+3>:    push    ebx
0x00001556 <+4>:    sub     esp,0x14
0x00001559 <+7>:    call    0x10d0 <__x86.get_pc_thunk.bx>
0x0000155e <+12>:   add     ebx,0x2a6e
0x00001564 <+18>:   mov     DWORD PTR [ebp-0x10],0x12c
0x0000156b <+25>:   mov     DWORD PTR [ebp-0xc],0x190
0x00001572 <+32>:   mov     eax,DWORD PTR [ebx+0x40]

```

실제 함수가 쓰는 스택의 크기와 이론적으로 쓰는 크기가 틀렸고,
return address를 출력하기에 이 과제가 스택에 관한 것이기에 무관하다 생각하여 하지않
았다.

5. 결과

```

===== Current Call Stack =====
5 : var_1 = 100    <=== [esp]
4 : func1 SFP     <=== [ebp]
3 : Return Address
2 : arg1 = 1
1 : arg2 = 2
0 : arg3 = 3
=====

```

func1 epilog 이후의 상황이며 func1을 부를 때 가져간 인자와 ret address, sfp, func1의
지역변수인 var1이 들어있음을 알 수 있다. esp는 스택의 최상단, ebp는 sfp를 가르키고 있
는 것을 볼 수 있다.

```

===== Current Call Stack =====
10 : var_2 = 200   <=== [esp]
9 : func2 SFP = 4  <=== [ebp]
8 : Retrun Address
7 : arg1 = 11
6 : arg2 = 13
5 : var_1 = 100
4 : func1 SFP
3 : Return Address
2 : arg1 = 1
1 : arg2 = 2
0 : arg3 = 3
=====

```

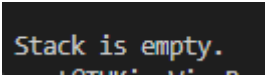
func2 epilog 이후의 상황이며 func2의 인자와 ret address, sfp, 지역변수 var2가 추가되었고 이전 func1의 스택 프레임은 유지되어있다.

```
===== Current Call Stack =====
15 : var_4 = 400    <=== [esp]
14 : var_3 = 300
13 : func3 SFP = 9    <=== [ebp]
12 : Return Address
11 : arg1 = 77
10 : var_2 = 200
9 : func2 SFP = 4
8 : Retrun Address
7 : arg1 = 11
6 : arg2 = 13
5 : var_1 = 100
4 : func1 SFP
3 : Return Address
2 : arg1 = 1
1 : arg2 = 2
0 : arg3 = 3
=====
```

func3 epilog 이후의 상황이며 func3의 인자, ret address, sfp, 지역변수 var3, var4가 추가되었고 이전 스택프레임들은 유지되고있다.

```
===== Current Call Stack =====
10 : var_2 = 200    <=== [esp]
9 : func2 SFP = 4    <=== [ebp]
8 : Retrun Address
7 : arg1 = 11
6 : arg2 = 13
5 : var_1 = 100
4 : func1 SFP
3 : Return Address
2 : arg1 = 1
1 : arg2 = 2
0 : arg3 = 3
=====
```

```
===== Current Call Stack =====
5 : var_1 = 100    <=== [esp]
4 : func1 SFP    <=== [ebp]
3 : Return Address
2 : arg1 = 1
1 : arg2 = 2
0 : arg3 = 3
=====
```



```
Stack is empty.
```

이후 이전과 똑같은 상태를 거치며 스택이 잘 정리됨을 알 수 있었다.